VELLORE INSTITUTE OF TECHNOLOGY, VELLORE · SCOPE

COMPILER DESIGN LABORATORY · BCSE307P

CodeFlow

A Domain-Specific Language and Compiler for Workflow Automation

Final Project Report

Phases 1, 2 & 3 — Design, Core Compiler Construction, Optimization & Execution

Name: Daksh Arora

Registration No.: 24BCI0259

Programme: B.Tech CSE (Information Security)

Project type: Individual Project

Implementation: TypeScript / Node.js

Date: October 29, 2026

GitHub: https://github.com/dakshcodez/CodeFlow_DSL

Contents

1. Introduction & Objectives
2. The CodeFlow Language
3. Compiler Architecture
4. Phase 1 — Lexer, Parser & AST
5. Phase 2 — Symbol Table, Semantic Analysis & TAC Generation
6. Phase 3 — Optimization & Execution Engine
7. Pipeline, CLI & Browser Visualization
8. End-to-End Worked Examples
9. Testing & Verification
10. Requirements Compliance, Limitations & Future Work
11. Conclusion

1. Introduction & Objectives

Many automation rules — “if the temperature exceeds 35, raise an alert and start the fan” — are written as ad-hoc configuration or scripts that are evaluated directly, with no static checking. **CodeFlow** is a small, statically typed, textual Domain-Specific Language for exactly this kind of conditional workflow logic, together with a real compiler and execution environment implemented in TypeScript. It is deliberately *not* a general-purpose language and is not presented as novel research or a production workflow platform; its purpose is to demonstrate every stage of a compiler on a deliberately constrained language.

The project was delivered in three phases:

Phase	Theme	Delivered
1	Problem definition & system design	27-page proposal, language specification, grammar, architecture; lexer, recursive-descent parser and AST prototype
2	Core compiler construction	Symbol table, semantic analyzer (scope + type checking), Three-Address Code generator, layered diagnostics
3	Optimization, execution & integration	Constant folding, unreachable- and dead-code elimination, TAC interpreter with runtime errors, full visualization, end-to-end tests

The complete implemented pipeline is:

```
Source → Lexer → Tokens → Recursive-Descent Parser → AST → Semantic Analyzer + Symbol Table → Three-Address Code → Optimizer → Optimized TAC → Execution Engine → Trace
```

Two concerns span the pipeline: a common `CompilerError` diagnostic model, and a browser-based Compiler Explorer that exposes every intermediate artifact.

2. The CodeFlow Language

2.1 Lexical Structure

Keywords: `workflow sensor input when otherwise action alert log number string boolean AND OR NOT true false`. **Identifiers:** `(letter|_)(letter|digit|_)*`, not a keyword. **Literals:** numbers (`35`, `2.5`), double-quoted strings, and `true` / `false`. The only types are `number`, `string` and `boolean`.

2.2 Grammar (EBNF)

```
program       ::= { workflowDecl }
workflowDecl  ::= "workflow" STRING "{" { declaration } { statement } "}"
declaration   ::= ("sensor" | "input") IDENT ":" type
type          ::= "number" | "string" | "boolean"
statement     ::= whenStmt | otherwiseStmt | actionStmt | alertStmt | logStmt
whenStmt      ::= "when" expression block
otherwiseStmt ::= "otherwise" block
block         ::= "{" { statement } "}"
actionStmt    ::= "action" IDENT "(" [ argList ] ")"
alertStmt     ::= "alert" STRING          logStmt ::= "log" STRING
argList       ::= expression { "," expression }
expression    ::= orExpr
orExpr        ::= andExpr { "OR" andExpr }
andExpr       ::= notExpr { "AND" notExpr }
notExpr       ::= [ "NOT" ] relExpr
relExpr       ::= addExpr [ relOp addExpr ]
relOp         ::= ">" | "<" | "⩾" | "⩽" | "⩵" | "≠"
addExpr       ::= mulExpr { ("+" | "-") mulExpr }
mulExpr       ::= unaryExpr { ("*" | "/") unaryExpr }
unaryExpr     ::= [ "-" ] primary
primary       ::= NUMBER | STRING | BOOLEAN | IDENT | "(" expression ")"
```

2.3 Operator Precedence

Level	Operators	Associativity	Parser function
1 (lowest)	`OR`	Left	`parseOr`
2	`AND`	Left	`parseAnd`
3	`NOT`	Right	`parseNot`
4	`> < ⩾ ⩽ ⩵ ≠`	Left	`parseRelational`
5	`+ -`	Left	`parseAdditive`
6	`* /`	Left	`parseMultiplicative`
7 (highest)	unary `-`	Right	`parseUnary`

2.4 Semantics in Brief

- A `workflow` is the only scope; `sensor` / `input` declarations are visible only inside it.
- A `when` condition must be `boolean`. An `otherwise` immediately following a `when` runs only if that condition was false.
- `action`, `alert` and `log` are observable runtime effects, not real external side effects. Sensor/input values are supplied at run time.

2.5 Canonical Programs

```
workflow "CoolingSystem" {
    sensor temperature : number

    when temperature > 35 {
        alert "High temperature"
        action start_fan()
    }

    otherwise {
        log "Temperature normal"
    }
}

workflow "OrderProcessing" {
    input orderAmount : number

    when orderAmount > 5000 {
        action fraud_check()
    }

    when orderAmount > 10000 {
        action manager_approval()
    }

    action process_payment()
}
```

2.6 Scope Boundaries

Functions, loops, recursion, arrays, objects, modules, concurrency, networking, persistence, authentication, register allocation and native code generation are explicitly out of scope, keeping the DSL small enough to finish and explain.

3. Compiler Architecture

Module	Location	Input → Output	Phase
Lexer	`src/lexer/`	source → `Token[]`	1
Parser	`src/parser/`	`Token[]` → AST	1
AST nodes	`src/ast/`	discriminated unions	1
Diagnostics	`src/diagnostics/`	common `CompilerError`	1 (extended 2, 3)

Module	Location	Input → Output	Phase
Symbol table	`src/symbols/`	per-workflow scope	2
Semantic analyzer	`src/semantic/`	AST → symbol tables + errors	2
TAC generator / printer	`src/ir/`	AST → `IRInstruction[]`	2
Optimizer	`src/optimizer/`	TAC → optimized TAC	3
Execution engine	`src/runtime/`	optimized TAC + inputs → trace	3
Pipeline, CLI, web UI	`src/compiler/`, `src/cli.ts`, `web/`	wiring & visualization	1–3

Design rules enforced throughout: stages communicate only through typed artifacts; the parser performs no symbol lookup or type checking; the optimizer never sees source text; the runtime executes TAC rather than reparsing source; source locations flow from tokens into AST nodes, IR instructions and diagnostics. The compiler is roughly 2,100 lines of TypeScript in `src/`, plus a 184-line browser front end and about 1,500 lines of tests.

4. Phase 1 — Lexer, Parser & AST

Phase 1 produced the 27-page proposal and design document (problem statement, motivation, objectives, scope, language specification, requirements, methodology, architecture, module description, technology stack) and a prototype covering *source → tokens → AST* with basic syntax validation.

4.1 Lexer

`src/lexer/lexer.ts` (245 lines) is a hand-written scanner producing `Token = { kind, lexeme, line, column, start, end }` with an explicit `TokenKind` enumeration, so the parser never compares raw strings. It skips whitespace, distinguishes keywords from identifiers, and recognizes numbers (integer and decimal), strings, booleans, operators and delimiters. It reports three classes of lexical error — *illegal characters*, *malformed numeric literals* and *unterminated strings* — each with line and column, without aborting the scan.

4.2 Recursive-Descent Parser

`src/parser/parser.ts` (436 lines) has one method per grammar production (`parseProgram`, `parseWorkflow`, `parseDeclaration`, `parseStatement`, `parseWhen`, `parseOtherwise`, `parseAction`, `parseAlert`, `parseLog`, then the expression chain of Section 2.3). Precedence and associativity are encoded structurally by the call chain rather than by ad-hoc tables. The grammar is LL(1), so one token of lookahead suffices. Syntax errors report what was expected, what was found, and where (e.g. *Expected '}' to close a block but found end of input*), and the parser keeps going after an error so several can be reported in one run.

4.3 AST

The AST (`src/ast/nodes.ts`, 112 lines) is a discriminated union tagged by `kind`, each node carrying a source location.

Node	Fields
`Program`	workflows
`Workflow`	name, declarations, statements
`Declaration`	declKind (sensor|input), name, type
`WhenStatement` / `OtherwiseStatement`	condition, body / body
`ActionStatement`	name, args
`AlertStatement` / `LogStatement`	message
`LiteralExpr` / `IdentifierExpr`	literalType, value / name
`UnaryExpr` / `BinaryExpr`	operator, operand(s)

5. Phase 2 — Symbol Table, Semantic Analysis & TAC

Phase 2 turned the syntax prototype into a semantically validated front end that emits an intermediate representation.

5.1 Symbol Table

`SymbolTable` (`src/symbols/symbolTable.ts`, 55 lines) records `name`, `kind` (sensor/input), `type` and `scope`, with `insert` and `lookup` operations. Because a workflow is the only scope boundary and declarations never nest, one flat `Map` per workflow is a correct and simple model; a declaration in one workflow is never visible in another. A duplicate `insert` is rejected and leaves the original entry intact.

5.3.2 Semantic Analyzer

`analyze(program)` (`src/semantic/analyzer.ts`, 217 lines) is the core new component of Phase 2. It performs a single recursive walk over the AST, enforcing every property the context-free grammar cannot express on its own: declaration checking, duplicate detection, scope checking, and a full static type-checking pass that assigns a `PrimitiveType` to every expression node.

5.2.1 Declaration & Scope Checking

Every `IdentifierExpr` is resolved against the enclosing workflow's `SymbolTable`. An identifier with no matching declaration is reported as `Undefined identifier '<name>'` at its source location. Because CodeFlow has exactly one scope level (the workflow), there is no scope-chain walk — resolution is a single `lookup()` call against the current workflow's table, and a declaration made in one workflow is never visible from another (verified by the *"does not leak a declaration from one workflow into another"* test).

5.2.2 Duplicate Declaration Checking

While populating each workflow's symbol table, a second declaration reusing an already-declared name is reported as `Duplicate declaration of '<name>' in workflow "<workflow>"`, and the original entry is left untouched in the table (so subsequent lookups still resolve against the first, valid declaration).

5.2.3 Type Checking

The analyzer performs bottom-up type inference over every `Expr`, returning a `PrimitiveType | undefined` (`undefined` marks an expression that already failed to type-check, which stops errors from cascading further up the same expression tree). Successfully typed expressions are recorded in `SemanticResult.types: Map<Expr, PrimitiveType>` for potential reuse by later stages.

Operator class	Operators	Required operand type(s)	Result type
Arithmetic	`+ - * /`	`number, number`	`number`
Relational	`> < ⩾ ⩽`	`number, number`	`boolean`

Operator class	Operators	Required operand type(s)	Result type
Equality	`== !=`	both operands the *same* primitive type	`boolean`
Logical (binary)	`AND OR`	`boolean, boolean`	`boolean`
Logical (unary)	`NOT`	`boolean`	`boolean`
Arithmetic (unary)	`-`	`number`	`number`

Additionally, the condition expression of every `when` statement is required to type as `boolean` (`The condition of a 'when' statement must be of type 'boolean', but found '<type>'`); a non-boolean condition such as `when temperature { ... }` is rejected even though `temperature` itself is a validly-declared identifier.

Accepted programs (excerpt from the test suite)

```
number > number        // relational, numeric operands
number + number        // arithmetic
boolean AND boolean    // logical
string == string       // equality, matching types
boolean == boolean     // equality, matching types
```

Rejected programs (excerpt from the test suite)

```
"hello" > 10           // relational requires number, number
true + 5               // arithmetic requires number, number
10 AND 5               // logical requires boolean, boolean
temperature AND 10     // temperature: number, AND requires boolean
```

5.2.4 Error Recovery

Analysis does not stop at the first diagnostic. Mirroring the parser's own recovery strategy, the analyzer keeps walking the remaining declarations, statements, and workflows after reporting an error, so a single compilation pass can surface every semantic problem in a program rather than forcing the user to fix and recompile one error at a time. Errors are still prevented from *cascading* within one expression: once an operand fails to resolve or type-check, the enclosing expression returns `undefined` instead of guessing a type and reporting a second, spurious error on top of the first.

5.2.5 Diagnostic Output

Every semantic diagnostic is emitted as a `CompilerError` with `stage: "semantic"` and the offending node's `line` / `column`, consistent with the shared cross-cutting error model used by the lexer and parser:

```
type CompilerError = {
  stage: "lexical" | "syntax" | "semantic" | "runtime";
  message: string;
  line?: number;
  column?: number;
  length?: number;
};
```

5.3 Three-Address Code Generation

5.3.1 Instruction Model

`src/ir/instructions.ts` defines TAC as a discriminated union (`IRInstruction`) of nine structured instruction shapes, rather than free-form strings, so the instruction stream stays type-safe and straightforward for the Phase 3 optimizer and execution engine to pattern match over:

```
type IRInstruction =
  | AssignInstruction            // dest = value
  | BinaryInstruction            // dest = left OP right
  | UnaryInstruction             // dest = OP operand
  | ConditionalJumpInstruction   // IF_FALSE cond GOTO label
  | JumpInstruction              // GOTO label
  | LabelInstruction             // LABEL label
  | CallInstruction              // CALL name(args)
  | AlertInstruction             // ALERT "message"
  | LogInstruction;              // LOG "message"
```

Operands are likewise a closed union:

```
type Operand =
  | { kind: "Temp";  id: number }               // compiler-generated temporary
  | { kind: "Var";   name: string }              // a declared sensor/input
  | { kind: "Const"; value: number | string | boolean }; // a literal value
```

The generator always materializes `Var` and `Const` operands into a `Temp` via an `Assign` instruction before they are used in any computation, so every instruction only ever references the result of at most one prior operation — true three-address discipline, with no instruction doing more than "one operator's worth" of work. Each workflow lowers to its own `WorkflowIR = { workflowName, instructions: IRInstruction[] }`.

5.3.2 Generation Algorithm

`generateIR(program)` (`src/ir/generator.ts`, 152 lines) lowers each workflow's AST independently — workflows never call one another in CodeFlow, so per-workflow temporary (`t1, t2, ...`) and label (`L1, L2, ...`) counters simply reset at the start of each workflow.

- **Expressions** always lower to a `Temp` operand: a `LiteralExpr` or `IdentifierExpr` is first assigned into a fresh temporary, then combined through `Binary` / `Unary` instructions whose operands are themselves temporaries.
- `when` **alone** lowers to: evaluate the condition, `IfFalse cond GOTO endLabel`, the body, then `Label endLabel`.
- `when` **immediately followed by** `otherwise` in the same statement list lowers to a structured if/else: `IfFalse cond GOTO elseLabel`, the `when` body, `Goto endLabel`, `Label elseLabel`, the `otherwise` body, `Label endLabel`. This pairing is purely structural — adjacency of the two statements in the list — and introduces no new grammar rule, matching the language's definition of `otherwise` as a fallback executed only when the immediately preceding `when`'s condition is false.
- **An orphaned** `otherwise` (no directly preceding `when`) has no condition to fall back on, so its body is lowered unconditionally.
- `action` / `alert` / `log` lower directly to `Call` / `Alert` / `Log` instructions; action arguments are lowered like any other expression and passed as a list of operands.

5.3.3 Worked Example

For the canonical example program `examples/cooling.cflow`:

```
workflow "CoolingSystem" {
    sensor temperature : number

    when temperature > 35 {
        alert "High temperature"
        action start_fan()
    }

    otherwise {
        log "Temperature normal"
    }
}
```

the generator emits (as rendered by the CLI / `formatWorkflowIR`):

```
workflow "CoolingSystem":
    t1 = temperature
    t2 = 35
    t3 = t1 > t2
    IF_FALSE t3 GOTO L1
    ALERT "High temperature"
    CALL start_fan()
    GOTO L2
LABEL L1
    LOG "Temperature normal"
LABEL L2
```

`src/ir/print.ts` (43 lines) is a pure formatter that renders this human-readable text from a `WorkflowIR`; it is consulted only by the CLI and the web visualization and is never read back by any compiler stage, preserving the rule that IR consumers act on the structured `IRInstruction[]`, not on its printed form.

6. Phase 3 — Optimization & Execution Engine

Phase 3 completed the pipeline: the TAC is now optimized and then actually executed.

6.1 Optimizer

`src/optimizer/optimizer.ts` (270 lines) implements three passes over each workflow's `IRInstruction[]`, composed by `optimizeWorkflow` in the order fold → unreachable → dead. Each pass is an exported, independently testable function.

Pass	What it does	Key design points
Constant folding `foldConstants`	Replaces `Binary` / `Unary` instructions whose operands are compile-time constants with a single `Assign` of the result (`10 + 20 → 30` , `5 > 3 → true`).	Constant knowledge is tracked per temporary and reset at every `Label` (a control-flow merge point). Sensor/input `Var` operands are never constant. Division by a constant zero is deliberately left unfolded so the runtime reports it.
Unreachable-code elimination `eliminateUnreachableCode`	Builds a small control-flow graph over instruction indices (fall-through plus `Goto` / `IfFalse` edges) and keeps only instructions reachable from the start by breadth-first search.	When an `IfFalse` condition is a known boolean (thanks to folding), only the edge it can actually take is followed, so a statically false `when` loses its body.
Dead-code elimination `eliminateDeadCode`	Removes `Assign` / `Binary` / `Unary` instructions whose result temporary is never read.	One backward pass suffices because CodeFlow has no loops: every jump target lies after its jump and each temporary has one static definition.

Register allocation, instruction scheduling and inter-procedural optimization are out of scope and not implemented.

Before / after: `examples/dead-code.cflow`

```
workflow "DeadCodeDemo" {
    sensor temperature : number
    when temperature > 10 + 20 { alert "High temperature" }
    when 5 > 3                 { log "Always runs" }
    when 2 > 9                 { action never_called() }
}
```

Generated TAC (20 instructions)	Optimized TAC (12 instructions)

`t1 = temperature`
`t2 = 10`
`t3 = 20`
`t4 = t2 + t3`
`t5 = t1 > t4`
`IF_FALSE t5 GOTO L1`
`ALERT "High temperature"`
`LABEL L1`
`t6 = 5`
`t7 = 3`
`t8 = t6 > t7`
`IF_FALSE t8 GOTO L2`
`LOG "Always runs"`
`LABEL L2`
`t9 = 2`
`t10 = 9`
`t11 = t9 > t10`
`IF_FALSE t11 GOTO L3`
`CALL never_called()`
`LABEL L3`
	`t1 = temperature`
	`t4 = 30`
	`t5 = t1 > t4`
	`IF_FALSE t5 GOTO L1`
	`ALERT "High temperature"`
	`LABEL L1`
	`t8 = true`
	`IF_FALSE t8 GOTO L2`
	`LOG "Always runs"`
	`LABEL L2`
	`t11 = false`
	`IF_FALSE t11 GOTO L3`
	`LABEL L3`

`10 + 20` folded to `t4 = 30` and the now-unused `t2`, `t3` (and `t6`, `t7`, `t9`, `t10`) were removed as dead code; the `CALL never_called()` body of the statically false `when 2 > 9` was removed as unreachable. Running it with `temperature = 40` yields `ALERT: High temperature` and `LOG: Always runs`, identical to the unoptimized program.

> **Known limitation.** A conditional jump on a known constant (`t8 = true; IF_FALSE t8 GOTO L2`) is not rewritten into an unconditional jump or removed; only the branch bodies that can never run are removed. This is semantically safe and the interpreter handles it, but it is a missed optimization.

6.2 Execution Engine

`src/runtime/interpreter.ts` (238 lines) interprets the optimized TAC of each workflow with a program counter, a map of temporaries, and a label index. It supplies sensor/input values from a user-provided `RuntimeInputs` record (CodeFlow has no real sensors or actuators).

Instruction	Runtime behaviour
`Assign`, `Binary`, `Unary`	Evaluate operands and store the result in the destination temporary
`IfFalse` / `Goto` / `Label`	Branch by setting the program counter from the label index; labels are no-ops
`Call`	Records an `Action` effect with evaluated arguments
`Alert` / `Log`	Records an `Alert` / `Log` effect

Execution produces a `WorkflowExecutionResult = { workflowName, trace, errors }`. Actions are observable effects (`ACTION: start_fan()`), not real side effects. Runtime errors halt the workflow, are recorded as a `"runtime"`-stage `CompilerError` with location, and the trace produced so far is preserved. Detected runtime errors are: a missing input value, a supplied value whose type differs from the declared type, division by zero, and operand type mismatches in operators.

7. Pipeline, CLI & Browser Visualization

7.1 Pipeline API

`src/compiler/pipeline.ts` (65 lines) exposes two functions:

```
compile(source)            // lex → parse → analyze → generateIR → optimize
  → { tokens, program, symbolTables, ir, optimizedIR, errors }
run(source, runtimeInputs) // compile(), then execute(optimizedIR, inputs,
symbolTables)
  → CompileResult + execution[]
```

Every stage runs over whatever the previous stage produced, even if earlier stages reported errors, so that all artifacts and as many diagnostics as possible are visible in one pass. Execution is a separate step because compiling never needs runtime values.

7.2 Command-Line Interface

```
npm run compile -- examples/cooling.cflow
npm run compile -- examples/cooling.cflow '{"temperature": 38}'
```

Prints tokens, AST (JSON), per-workflow symbol tables, TAC, optimized TAC, diagnostics, and — if a JSON object of inputs is given — the execution trace.

7.3 Compiler Explorer (Browser)

The web interface (`web/index.html`, `styles.css`, `app.ts`) is a lightweight, dependency-free page bundled with esbuild. It shows Source, Runtime Inputs (JSON), Tokens, AST, Symbol Table, Three-Address Code, Optimized Three-Address Code, Errors and Execution Trace side by side. It exists to make compiler internals observable, not to be a workflow dashboard.

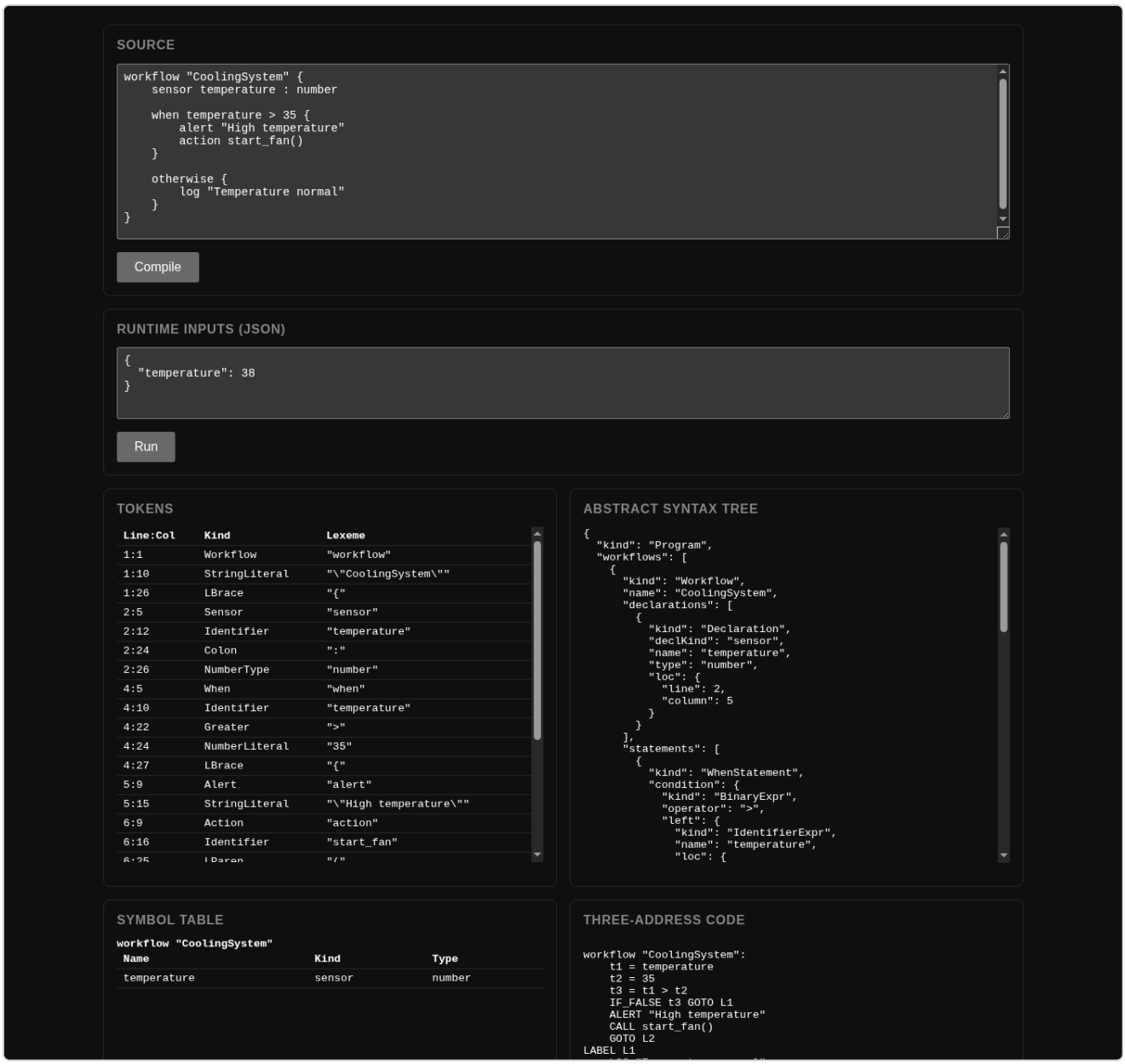


Figure 1 — Compiler Explorer running the CoolingSystem example with `temperature = 38` (upper part of the page; screenshot of the actual build).

8. End-to-End Worked Examples

8.1 CoolingSystem

Symbol table: `temperature sensor number`. **Generated TAC** (the optimizer finds nothing to change here, so the optimized TAC is identical):

```
workflow "CoolingSystem":
    t1 = temperature
    t2 = 35
    t3 = t1 > t2
    IF_FALSE t3 GOTO L1
    ALERT "High temperature"
    CALL start_fan()
    GOTO L2
LABEL L1
    LOG "Temperature normal"
LABEL L2
```

Inputs	Execution trace
`{"temperature": 38}`	`ALERT: High temperature`, `ACTION: start_fan()`
`{"temperature": 20}`	`LOG: Temperature normal`

8.2 OrderProcessing

```
t1 = orderAmount        t4 = orderAmount
t2 = 5000               t5 = 10000
t3 = t1 > t2            t6 = t4 > t5
IF_FALSE t3 GOTO L1     IF_FALSE t6 GOTO L2
CALL fraud_check()      CALL manager_approval()
LABEL L1                LABEL L2
                        CALL process_payment()
```

With `{"orderAmount": 12000}` the trace is `ACTION: fraud_check()`, `ACTION: manager_approval()`, `ACTION: process_payment()`.

8.3 Runtime errors

Program / inputs	Diagnostic
`when a / b > 2 { ... }` with `{"a":4,"b":0}`	`Runtime error at line 4, column 10: Division by zero`
CoolingSystem with `{"temperature":"hot"}`	`Runtime error at line 4, column 10: Type mismatch for 'temperature': expected number but received string ("hot")`

8.4 Compile-time errors (one per stage)

File in `examples/errors/`	Diagnostic produced
`invalid-token.cflow`	`Lexical error at line 4, column 5: Illegal character '@'`

File in `examples/errors/`	Diagnostic produced
`missing-brace.cflow`	`Syntax error at line 7, column 1: Expected '}' to close a block but found end of input` (and a second for the workflow body)
`undefined-identifier.cflow`	`Semantic error at line 4, column 10: Undefined identifier 'temperature'`
`invalid-type.cflow`	`Semantic error at line 6, column 10: Operator '+' expects operands of type 'number, number', but found 'boolean' and 'number'`

9. Testing & Verification

Tests are written with Vitest and mirror the compiler stages. The full suite currently passes: **10 test files, 131 tests**, and the project type-checks with `tsc --noEmit`.

Suite	Tests	Covers
`tests/lexer`	12	keywords, identifiers, numbers, strings, operators, invalid characters, malformed numbers, unterminated strings, line/column tracking
`tests/parser`	16	workflows, declarations, when/otherwise, nesting, actions, precedence, parentheses, malformed braces/parentheses, missing expressions
`tests/symbols`	6	insert, lookup, duplicates, scope isolation
`tests/semantic`	22	undefined identifiers, duplicates, numeric/boolean/equality typing, rejected type combinations, workflow scope
`tests/ir`	14	arithmetic, comparisons, conditional jumps, labels, calls, alerts, logs, multiple conditions
`tests/optimizer`	20	each pass as input-IR → output-IR, folding edge cases (e.g. division by zero), unreachable and dead code, and unchanged execution semantics
`tests/runtime`	14	true/false branches, multiple conditions, action dispatch, alerts, logs, missing input, type mismatch, division by zero
`tests/compiler`	10	pipeline wiring and the required malformed programs
`tests/e2e`	9	full source → trace regression for representative and malformed programs
`tests/web`	8	browser UI rendering (jsdom)
Total	**131**	

10. Requirements Compliance, Limitations & Future Work

10.1 Definition of Done

Requirement	Status	Implemented in
Lexed, with location tracking	Done	`src/lexer/`
Parsed by hand-written recursive descent; AST built	Done	`src/parser/`, `src/ast/`
Semantically validated; symbol table populated	Done	`src/semantic/`, `src/symbols/`
TAC generated as structured instructions	Done	`src/ir/`
TAC optimized (fold, unreachable, dead code), before/after visible	Done	`src/optimizer/`, UI panels
Optimized TAC executed; trace produced	Done	`src/runtime/`
Errors reported consistently by stage, with locations	Done	`src/diagnostics/`
Artifacts visible in the UI	Done	`web/`

10.2 Limitations

- Constant-condition branches are not collapsed into unconditional jumps (Section 6.1).
- Constant folding is local between labels; constants are not propagated across merge points.
- Actions are simulated effects only; there are no user-defined actions, integrations or real sensors by design.
- Each workflow is compiled and executed independently, and all workflows receive the same input record.
- The browser UI is intentionally minimal and has no editor features such as inline error highlighting.

10.3 Possible Future Work

- Rewrite constant-condition `IfFalse` into `Goto`/nothing for a fully collapsed IR.
- Per-instruction highlighting that links an IR line or trace entry back to its source line in the UI.
- A step-through execution view showing temporaries after each instruction.

11. Conclusion

CodeFlow demonstrates a complete compiler pipeline for a small, domain-specific language. Phase 1 fixed the language, grammar and architecture and delivered the lexer, recursive-descent parser and AST. Phase 2 added the scoped symbol table, a type-checking semantic analyzer and a structured Three-Address Code generator. Phase 3 added constant folding, unreachable- and dead-code elimination, a TAC interpreter with runtime error handling, a complete Compiler Explorer, and end-to-end regression tests. Every transformation is real, consumed by the next stage, and inspectable: the optimizer's effect is visible as before/after IR, and execution runs on the optimized IR rather than on the source.